MANUAL DE INSTALACIÓN DEL SOFTWARE

LAVA RÁPIDO VEHICULAR

Guía de preparación, instalación, configuración, verificación y despliegue

José Stiven Perdomo Martínez

Santiago Gordo Pérez

Servicio Nacional de Aprendizaje – SENA

Tecnólogo en Análisis y Desarrollo de Software

Ficha 3145555

Colombia · 2026

Resumen

Este manual establece un procedimiento reproducible para preparar y ejecutar Lava Rápido Vehicular en ambientes locales y controlados, y ofrece criterios para su publicación. Comprende PostgreSQL y Liquibase, el backend Java/Spring Boot, el panel administrativo React/Vite, la aplicación móvil Expo/React Native, integraciones Wompi, correo y notificaciones. Las instrucciones separan configuración, ejecución y comprobaciones, sin publicar credenciales ni datos personales. Se toma como referencia el estado técnico documentado al 4 de noviembre de 2026, que no equivale a certificar el funcionamiento actual de todos los proveedores.

Palabras clave: instalación de software, Spring Boot, PostgreSQL, Liquibase, React, Expo, Render.

Control documental

Campo	Valor
Documento	Manual de instalación
Proyecto	Lava Rápido Vehicular
Edición	1.0 · 4 de noviembre de 2026
Público	Desarrolladores, responsables de despliegue y evaluadores
Ambientes	Desarrollo, QA/sandbox y publicación
Advertencia	No ejecutar migraciones sobre bases reales sin respaldo ni autorización

Contenido

1. Presentación, alcance y precauciones

2. Arquitectura y requisitos de instalación

3. Preparación del entorno y repositorios

4. Instalación de PostgreSQL y Liquibase

5. Instalación del backend Spring Boot

6. Instalación del panel administrativo

7. Instalación de la aplicación móvil

8. Integraciones externas y seguridad

9. Publicación y despliegue

10. Verificación de instalación

11. Problemas frecuentes y recuperación

12. Evidencias, lista de cierre y referencias

1. Presentación, alcance y precauciones

Lava Rápido Vehicular es una plataforma de autolavado con aplicación móvil para clientes y operadores, panel web administrativo y API REST compartida. El objetivo de este manual es permitir a otro integrante del equipo instalar sus componentes, comprobar sus dependencias y reconocer fallas habituales sin alterar datos productivos.

1.1 Alcance

- Instalación local de los componentes de aplicación y de la base de datos.
- Configuración de ambientes de pruebas y preparación de una publicación segura.
- Compilación del panel web y generación de una build Android, previa validación del perfil EAS.
- Comprobaciones posteriores y registro de evidencias reproducibles.

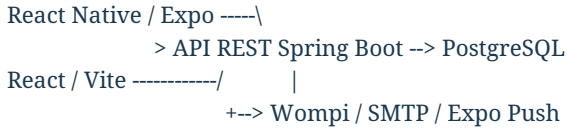
No cubre seguimiento GPS de operadores, optimización de rutas, inventario físico ni contabilidad detallada. Los puntos mostrados en mapas no son seguimiento de operarios.

1.2 Reglas antes de iniciar

- No usar credenciales reales en ejemplos, capturas, comandos versionados o el documento.
- Los comandos de instalación pueden crear dependencias, imágenes, contenedores y archivos de compilación; ejecutarlos únicamente en el entorno autorizado.
- Nunca aplicar cambios Liquibase sobre producción sin identificar changelog, versión, respaldo restaurable y ventana de mantenimiento.
- La evidencia de que existe código o una URL no demuestra por sí sola que toda la integración funcione actualmente.

2. Arquitectura y requisitos de instalación

La solución sigue una arquitectura cliente-servidor. Dos frontends consumen un backend monolítico modular Spring Boot, que concentra reglas, seguridad, persistencia e integración con proveedores. PostgreSQL solo es accesible desde el backend.



Componente	Tecnología declarada	Requisito de preparación
Base de datos	PostgreSQL 16; Liquibase 5.0.2 en	Docker y Docker Compose; conexión

	entorno Compose	aislada
Backend	Java 17; Spring Boot 4.0.9; Maven Wrapper	JDK 17 y puerto libre
Administración web	React 19; Vite 6; TypeScript	Node y npm compatibles con lockfile
Móvil	Expo ~58.0.20; React Native 0.88; TypeScript	Node, npm, Expo; EAS para compilaciones
Pagos/notificaciones	Wompi, SMTP y Expo Push	Cuentas sandbox y variables por ambiente

Las versiones anteriores son declaraciones del estado técnico documentado en noviembre de 2026, no una garantía de compatibilidad futura. Consultar manifests y archivos de bloqueo antes de instalar.

2.1 Ambientes

Ambiente	Propósito	Datos y controles
Desarrollo	Ejecutar servicios en el equipo local	Base aislada y datos ficticios
QA/sandbox	Pruebas integradas y pagos de prueba	Instancias de ensayo y evidencias depuradas
Producción	Uso final de la plataforma	Revisiones aprobadas, HTTPS, respaldo y monitoreo

3. Preparación del entorno y repositorios

3.1 Herramientas previas

1. Instalar Git, Java JDK 17, Docker/Compose y Node.js/npm compatibles con cada proyecto.
2. Confirmar que los puertos elegidos no están ocupados y disponer de acceso a Internet para dependencias.
3. Disponer de una terminal PowerShell (Windows) o shell equivalente y de permisos sobre los directorios del proyecto.
4. Identificar repositorios y registrar los SHA de las revisiones que se instalarán.

```
git --version
java -version
node --version
npm --version
docker --version
docker compose version
```

3.2 Ubicación de referencia

```
C:\Frontend-lavarapido\lava-rapido-admin
C:\Frontend-lavarapido\lava-rapido-mobile
```

```
C:\backend\backend-vehicular
C:\lavarapido-base-datos
```

Estas rutas son ejemplos documentados del equipo de desarrollo; otro equipo puede clonar los repositorios en directorios diferentes. No se inventan enlaces de clonación ni se incluyen secretos.

```
git status --short
git branch --show-current
git rev-parse HEAD
```

Registrar la versión sin modificar las ramas. Como referencia histórica, el frontend figuraba en HU-88-dev, el backend en develop y la base en main; la revisión efectiva de instalación debe determinarse de nuevo.

4. Instalación de PostgreSQL y Liquibase

4.1 Preparar base local

5. Abrir el directorio del repositorio de datos e inspeccionar docker-compose.yml antes de ejecutarlo.
6. Revisar puertos, volumen persistente, red y nombres de variables sin mostrar contraseñas.
7. Crear una configuración privada para el entorno local de acuerdo con el Compose; no subirla a Git.
8. Levantar solamente PostgreSQL mediante el servicio identificado en el Compose actual.
9. Esperar a que indique estado saludable y comprobar conectividad con un cliente autorizado.

```
cd C:\lavarapido-base-datos
docker compose config --services
docker compose up -d <nombre_servicio_postgres>
docker compose ps
```

Reemplazar el marcador del servicio por el nombre obtenido en docker compose config --services. El marcador es una instrucción, no un comando literal para copiar. Evitar docker compose down -v: puede eliminar los datos persistentes.

4.2 Migraciones y comprobación

10. Distinguir instalación nueva de actualización de una base ya existente.
11. Inspeccionar changelog-master.yaml y changelog-production.yaml; seleccionar la raíz aprobada para el ambiente.
12. En una base existente, guardar respaldo y verificar DATABASECHANGELOG y checksums antes de migrar.
13. Ejecutar Liquibase validate y status mediante el servicio de tooling o CLI definido por el repositorio.

14. Solo después de aprobación, ejecutar update contra la base correcta y comprobar las tablas y restricciones.

```
docker compose --profile tooling config --services
# Consultar comandos y perfil de Liquibase en el Compose actual
# Operaciones orientativas una vez configurada la conexión:
liquibase validate
liquibase status
liquibase update
```

El comando update modifica el esquema. La documentación del proyecto reporta 24 changesets activos y 18 tablas de aplicación declaradas, pero no certifica que estén aplicados en una base remota. No ejecutar un changelog histórico diferente sin comprobar su finalidad.

4.3 Criterios de éxito

- Contenedor de PostgreSQL saludable y conexión autenticada de backend con permisos mínimos.
- Historial DATABASECHANGELOG consistente con la raíz autorizada.
- Tablas esenciales presentes, con restricciones de reservas, pagos, operadores y notificaciones.
- Ausencia de pérdida de datos y respaldo probado antes de actualizar una instancia existente.

5. Instalación del backend Spring Boot

5.1 Preparar configuración

El backend utiliza Java 17 y Maven Wrapper. Sus parámetros se suministran por perfiles o variables de entorno; los nombres deben confirmarse contra los archivos application*.properties y Compose de la revisión elegida. No copiar valores de servidores compartidos a esta guía.

Grupo	Ejemplos de nombres declarados/esperados	Uso
Base de datos	DB_HOST, DB_PORT, DB_NAME, DB_USERNAME, DB_PASSWORD	Conexión PostgreSQL; comprobar esquema de variables real
Seguridad	JWT_SECRET, ADMIN_USER_IDS	Firma y políticas de identidad
Clientes/CORS	APP_FRONTEND_URL, APP_CORS_ALLOWED_ORIGINS	Orígenes web autorizados
Correo	MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD	Recuperación de contraseña
Pagos	WOMPI_ENVIRONMENT y claves correspondientes	Integración de sandbox/producción
Notificaciones	APP_NOTIFICACIONES_PUSH_ENABLED	Entrega push, cuando esté habilitada

5.2 Compilar y ejecutar

```
cd C:\backend\backend-vehicular
.\mvnw.cmd -version
.\mvnw.cmd test
.\mvnw.cmd package
.\mvnw.cmd spring-boot:run
```

Estos comandos requieren variables y base de datos configuradas. En Linux/macOS utilizar ./mvnw. Si el proyecto necesita un perfil Spring específico, activarlo por un mecanismo seguro y documentado. No asumir que un error de conexión implica una falla de código.

5.3 Verificar API

```
curl.exe -i http://localhost:<puerto_backend>/actuator/health/readiness
```

Sustituir el puerto por el configurado; el entorno histórico de pruebas usó 8081, mientras que un despliegue en Render puede usar PORT. La respuesta esperada cuando está listo es HTTP 200 con estado UP, si el endpoint está habilitado.

- Confirmar que los endpoints protegidos rechazan solicitudes sin autenticación.
- Comprobar que JPA valida el esquema sin intentar crear tablas de producción.
- Verificar logs depurados, sin contraseñas, tokens ni información personal.

6. Instalación del panel administrativo

15. Ubicarse en lava-rapido-admin y revisar package.json, scripts y lockfile.
16. Ejecutar npm ci para instalar exactamente las versiones bloqueadas, cuando el lockfile sea compatible.
17. Configurar VITE_API_URL según la estrategia real de desarrollo y publicación; verificar si ya contiene /api.
18. Ejecutar el servidor de desarrollo y comprobar la pantalla de autenticación.
19. Ejecutar los scripts existentes de prueba y compilación y validar dist.
20. Configurar fallback SPA en el alojamiento para rutas como /dashboard/panel y /dashboard/servicios.

```
cd C:\Frontend-lavarapido\lava-rapido-admin
npm ci
npm run
npm run dev
npm run build
```

En desarrollo se utiliza la estrategia de proxy de Vite cuando así esté configurada; la compilación publicada debe apuntar a la API HTTPS autorizada. Verificar RequireAdmin, 401 y 403 con cuentas ficticias.

7. Instalación de la aplicación móvil

7.1 Ejecución de desarrollo

21. Ingresar al directorio lava-rapido-mobile y confirmar manifest y lockfile.
22. Ejecutar npm ci y consultar los scripts disponibles en package.json.
23. Configurar EXPO_PUBLIC_API_URL con la API accesible desde el dispositivo o emulador; localhost del PC no siempre corresponde al dispositivo.
24. Iniciar Expo con el comando del proyecto y comprobar navegación de cliente y operador.
25. Validar permisos de red y comportamiento de mapas/avisos según el tipo de compilación.

```
cd C:\Frontend-lavarapido\lava-rapido-mobile
npm ci
npm run
npx expo start
```

7.2 Compilar Android para distribución

26. Inspeccionar app.json, app.config.js y eas.json para determinar perfiles y permisos.
27. Autenticar EAS con una cuenta autorizada, sin capturar ni compartir credenciales.
28. Generar una build Android con el perfil vigente; descargar el artefacto desde el resultado oficial.
29. Instalar la build en un dispositivo de prueba compatible y validar login, navegación, reservas y conexiones.
30. Registrar ID de build, versión Expo y SHA del frontend, sin confundir APK anterior con versión actual.

```
eas build --platform android --profile preview
eas build:list --platform android
```

Ejecutar la compilación solo si el perfil preview existe y se encuentra configurado para el proyecto. La build histórica de octubre utilizaba Expo SDK 57, mientras el manifest de noviembre declaró SDK 58; no atribuirla automáticamente a esta versión.

8. Integraciones externas y seguridad

8.1 Wompi

Para verificar pagos utilizar primero un ambiente sandbox con reservas y cuentas ficticias. El backend genera referencia e integridad y confirma con consulta servidor-servidor o webhook firmado. La devolución visual del widget no equivale a pago aprobado. Comprobar estados pendiente, aprobado, rechazado y prevención de duplicados.

8.2 Correo

La recuperación de contraseña depende de SMTP correctamente configurado. Verificar en QA solicitud de restablecimiento, expiración y uso único del enlace. En una comprobación histórica de Render el health general falló por timeout SMTP, aunque readiness respondió UP; se requiere una comprobación nueva para la instalación actual.

8.3 Avisos

Las notificaciones dentro de la aplicación tienen consulta y estado de lectura. Las entregas externas mediante Expo Push dependen de habilitación, permisos del dispositivo y configuración de proveedor. No afirmar que funcionan en producción sin prueba de entrega.

8.4 Seguridad de publicación

- Forzar HTTPS para URLs públicas y validar orígenes CORS específicos.
- Separar secretos del código, capturas y registros; evitar guardar credenciales en repositorios.
- No dar acceso a PostgreSQL directamente desde los clientes.
- Revisar roles, propiedad de recursos y políticas de reinicio de sesión.

9. Publicación y despliegue

9.1 Secuencia recomendada

31. Identificar commits y generar respaldo consistente de datos.
32. Validar changelog y aplicar cambios aprobados en ambiente correspondiente.
33. Publicar backend, revisar variables protegidas y comprobar readiness.
34. Construir el panel con API pública correcta y publicar como aplicación SPA.
35. Generar y distribuir build Android con EAS, y comprobar su conexión con la API.

36. Ejecutar pruebas de aceptación, generar evidencias y registrar decisión de entrega.

9.2 Direcciones reportadas

Servicio	Dirección pública documentada	Comprobación
Backend	https://lava-rapido-backend.onrender.com	Readiness histórico 200 UP; disponibilidad actual por comprobar
Panel	https://lava-rapido-admin.onrender.com	Capturas de interfaz proporcionadas; salud actual no verificada
APK Android	Distribución mediante EAS Build	Identificar artefacto correspondiente a la versión instalada

Las direcciones anteriores son referencias reportadas del proyecto, no evidencia de una verificación actual realizada al elaborar este manual. Adjuntar para la entrega capturas del panel de servicios de Render, estado, commit, fecha y respuesta de salud, con datos privados ocultos.

9.3 Publicación en Render: puntos a revisar

- Servicio backend: tipo de ejecución (Docker o comando), variables, puerto, estado de logs y enlace a versión publicada.
- Panel: directorio de salida dist, reglas de redirección SPA, orígenes de API y dominio HTTPS.
- Base: alojamiento autorizado, conectividad privada y política de respaldos y recuperación.
- Monitoreo: health general versus readiness, avisos de errores y seguridad de configuración.

10. Verificación de instalación

Caso	Acción	Criterio de aceptación
INS-01	docker compose ps; conexión a base	PostgreSQL saludable y esquema coherente
INS-02	GET readiness de API	HTTP 200 UP cuando lista
INS-03	Abrir panel por navegador	Carga login y rutas autorizadas
INS-04	Autenticación negativa	JWT ausente o inválido es rechazado
INS-05	Crear reserva ficticia en QA	Respeta horario y evita solapamientos
INS-06	Asignar operador de prueba	Permisos y transiciones consistentes
INS-07	Validar pago sandbox	Estado confirmado por backend
INS-08	Ejecutar build Android	Instala y consume API correcta
INS-09	Consultar notificaciones	Lecturas/contador reflejan datos persistidos

Esta tabla es un plan de comprobación; ningún caso nuevo se declara aprobado sin fecha, ambiente, resultado y evidencia vinculada a la versión instalada.

11. Problemas frecuentes y recuperación

Síntoma	Causa a comprobar	Acción segura
Backend no inicia	JDK, variables, BD y perfil	Consultar logs depurados; verificar JDK 17 y conexión
Error al validar esquema	Liquibase/JPA desalineados	Revisar changelog y DATABASECHANGELOG; no editar historia aplicada
Panel muestra error API	CORS, VITE_API_URL, prefijo /api	Comprobar URL, proxy, HTTPS y respuesta HTTP
Ruta del dashboard da 404	Hosting sin fallback SPA	Configurar reescritura a index.html
Móvil no conecta	URL localhost o red no accesible	Usar host accesible al dispositivo y HTTPS público
Wompi queda pendiente	Webhook/consulta o timeout	Consultar estado remoto; no forzar aprobación
Correo no llega	SMTP, firewall o credenciales	Ensayar en ambiente de pruebas; revisar salud
Push no llega	Bandera deshabilitada o dispositivo	Confirmar perfil nativo, token y logs de entrega

11.1 Reversión ante fallas críticas

37. Registrar versión afectada, error, fecha y alcance antes de modificar nada.
38. Detener publicación de nuevas versiones y recuperar revisión de aplicación aprobada.
39. Si la falla afectó datos, evaluar restauración desde un respaldo probado y cambios posteriores.
40. Repetir readiness y los casos mínimos de autenticación, reservas y pagos en ambiente controlado.
41. Documentar causa y acción correctiva para el seguimiento posterior.

12. Evidencias, cierre y referencias

12.1 Evidencias para anexar a la entrega

Código	Evidencia	Estado con material disponible
EV-01	Versiones, SHA y salida de compilaciones	Registrar al ejecutar esta instalación
EV-02	Estado PostgreSQL y Liquibase (sin credenciales)	Pendiente de ejecutar en ambiente elegido
EV-03	Captura de backend en Render y respuesta HTTP	URL y antecedente histórico; captura actual pendiente
EV-04	Captura de panel publicado y ruta interna	Capturas de UI anteriores disponibles
EV-05	EAS build actual y prueba en Android	Evidencia de revisión específica pendiente
EV-06	Casos de aceptación y errores corregidos	Pendiente de ejecución documentada
EV-07	Plan de respaldo/reversión y responsable	Requiere validación para producción

12.2 Lista final de control

- Ramas, commits y versiones identificadas.
- PostgreSQL saludable, respaldo y changelog aprobado.
- API inicia, protege endpoints y devuelve readiness correcto.
- Panel funciona con rutas internas y API configurada.
- Android instalado y conectado a la API correspondiente.
- Integraciones externas ensayadas en sandbox sin datos reales.
- Capturas, logs y acta de resultados depurados.

12.3 Conclusiones

La instalación debe proceder en orden: herramientas, base PostgreSQL/Liquibase, backend, panel y app móvil. El sistema dispone de una estructura apta para este procedimiento, pero la aceptación final depende de ejecutar las comprobaciones en la revisión concreta y conservar sus resultados. El manual es una guía reproducible, no una afirmación de que todas las verificaciones se hayan repetido durante su redacción.

Referencias

Equipo de Lava Rápido Vehicular. (2026). Procedimiento de implementación del software [Documento interno].

Equipo de Lava Rápido Vehicular. (2026). Manual técnico del sistema [Documento interno].

Equipo de Lava Rápido Vehicular. (2026). Especificación de requisitos de software [Documento interno].

Expo. (s. f.). Expo documentation. https://docs.expo.dev/

Liquibase. (s. f.). Liquibase documentation. https://docs.liquibase.com/

Render. (s. f.). Render documentation. https://render.com/docs

Spring. (s. f.). Spring Boot reference documentation. https://docs.spring.io/spring-boot/

Wompi. (s. f.). Documentación para desarrolladores. https://docs.wompi.co/